

Session 6

Exceptions

By : Rajashekar G.S
Java Full Stack Trainer

Review

- ❑ Inheritance allows the creation of hierarchical classifications.
- ❑ Inheritance allows the reusability of code.
- ❑ All methods and properties of the base class are inherited by objects of the derived class except the constructors.
- ❑ Polymorphism is the ability of different objects to respond to the same message in different ways.
- ❑ Overloaded methods are examples of static polymorphism and overridden methods are examples of dynamic polymorphism.
- ❑ Access specifiers are used to determine how the class members are accessed.
- ❑ Some of the method modifiers in Java are `static`, `final`, `abstract`.

Objectives

- ❑ *Define Exception*
- ❑ *Explain exception handling*
- ❑ *Describe the try, catch and finally blocks*
- ❑ *Examine multiple catch blocks*
- ❑ *Explore nested try/catch blocks*
- ❑ *Explain the use of throw and throws keywords*
- ❑ *Create user defined exceptions*

Exception

- ❑ Exception is an abnormal condition.
- ❑ Exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.
- ❑ An exception is a problem that arises during the execution of a program.
- ❑ An exception is an unexpected event that occurs during program execution.
- ❑ It affects the flow of the program instructions which can cause the program to terminate abnormally.

Exceptions

- ❑ An exception can occur for many reasons. Some of them are:
- ❑ Invalid user input
- ❑ Device failure
- ❑ Loss of network connection
- ❑ Physical limitations (out of disk memory)
- ❑ Code errors
- ❑ Opening an unavailable file

What is an exception?

```
class ExceptionRaised {  
    /** Constructor. */  
    protected ExceptionRaised() {  
    }  
    /**  
     * This method calculates the remainder of the division.  
     * @param operand1 is operator in division  
     * @param operand2 is denominator in division  
     * @return remainder of the division.  
     */  
    static int calculateRemainder(final int operand1, final int operand2) {  
        int result = operand1 / operand2; // user defined method  
        return result;  
    }  
    /**  
     * Sole entry point to the class and application.  
     * @param args Array of String arguments  
     */  
    public static void main(String[] args) {  
        // The variable result is defined to store the result.  
        int result = calculateRemainder(9, 0);  
        System.out.println(result);  
        try {  
            // Exception object  
            Exception e = new Exception("Exception occurred");  
            e.printStackTrace();  
        }  
    }  
}
```

- Can be generated manually in a program
- OR
- Generated by Java Runtime

Abnormal Condition

Exception

Error Handling Benefits

- Fixes Error
- Prevents automatic termination

Program Terminates Abruptly and control is given to OS

OS

Handling Exceptions 2-1

A pseudo code handling a runtime error

.....

IF B IS ZERO GO TO ERROR

C = A / B

PRINT C

GO TO EXIT

ERROR:

BLOCK THAT
HANDLES THE
EXCEPTION

{ "CODE CAUSING ERROR DUE TO DIVISION BY ZERO"

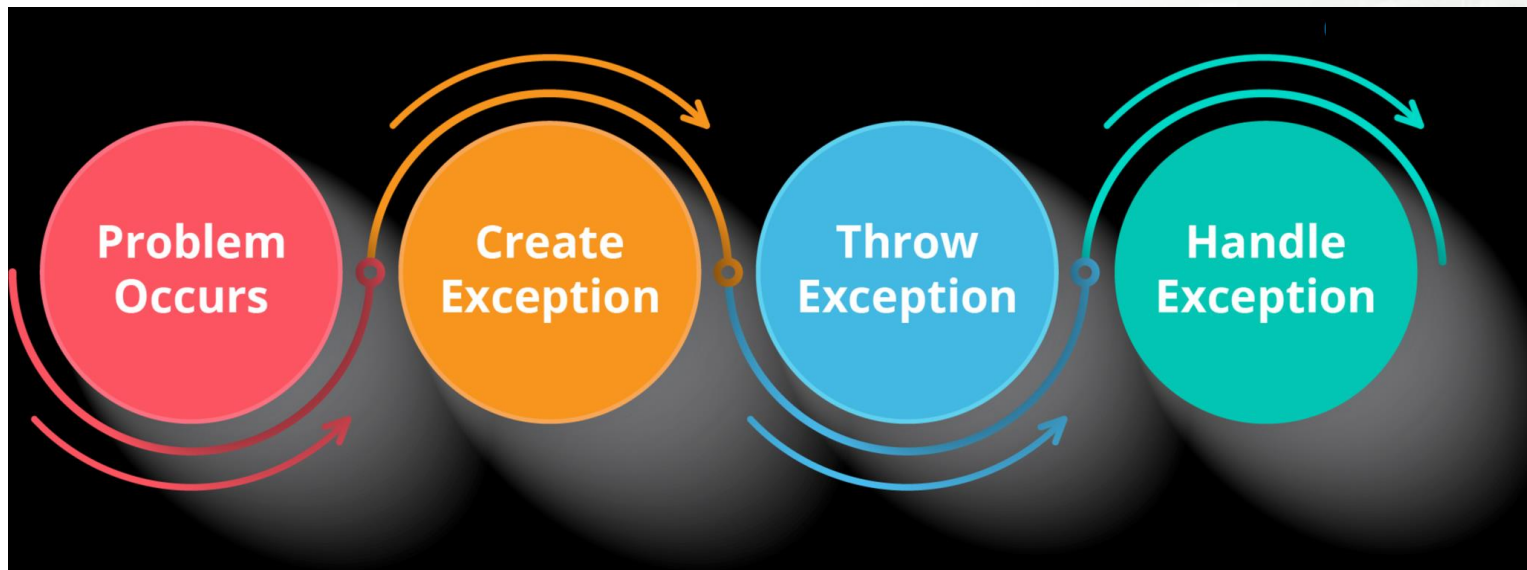
DISPLAY

EXIT:

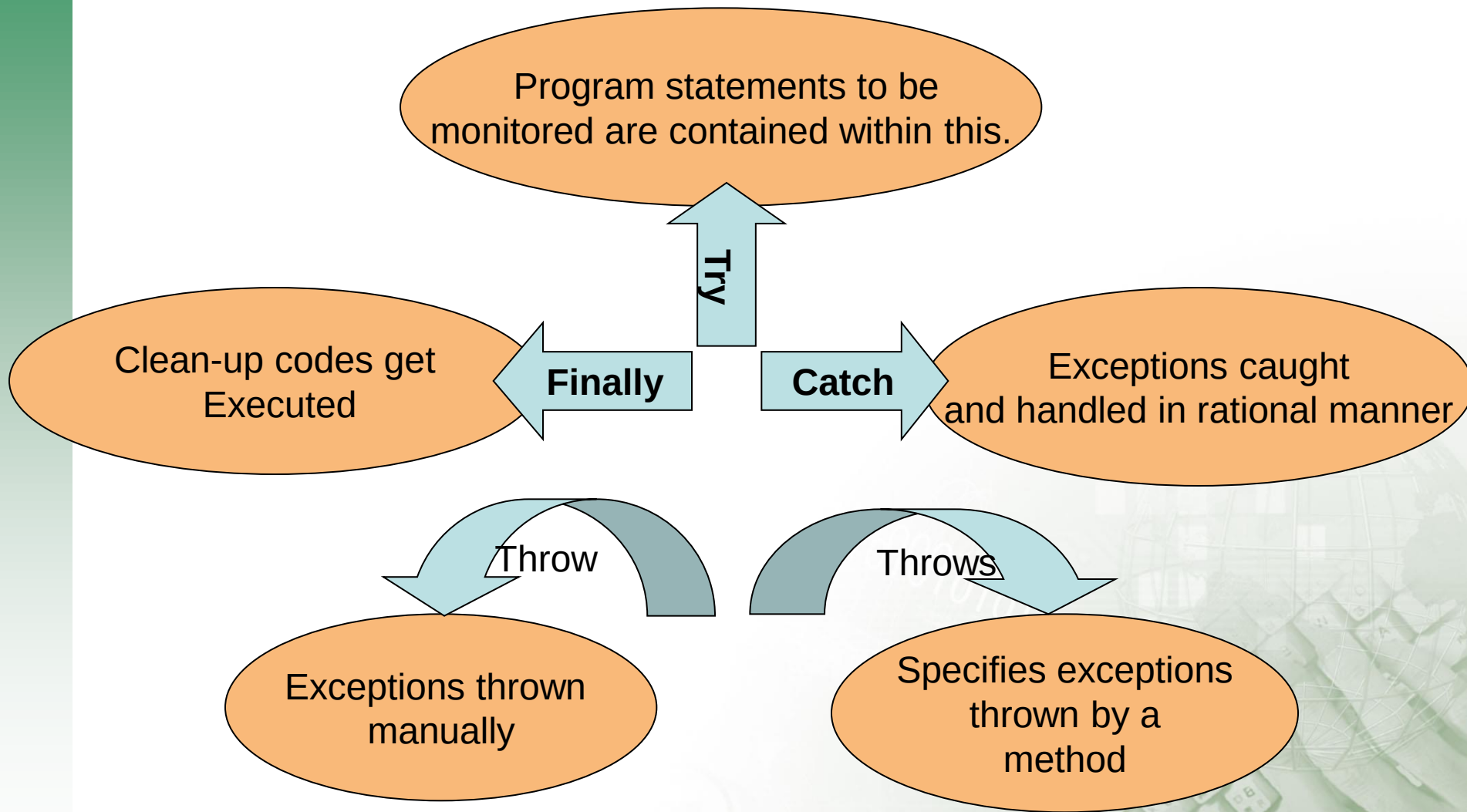
END

Exception Handling

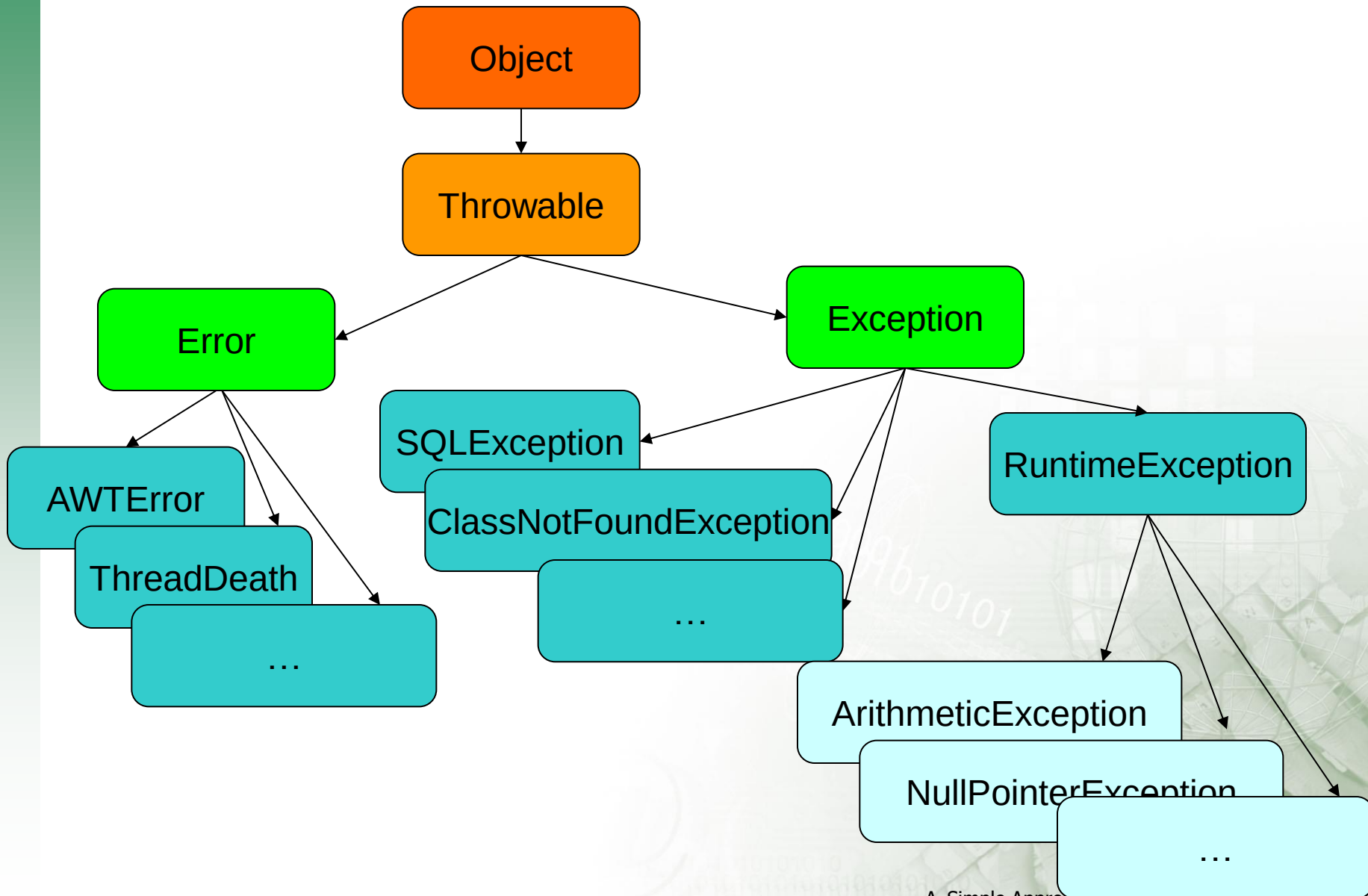
- ❑ The Exception Handling in Java is one of the powerful mechanism to handle the runtime errors so that normal flow of the application can be maintained.
- ❑ If an exception is not handled, it may lead to a system failure.
- ❑ That is why handling an exception is very important.



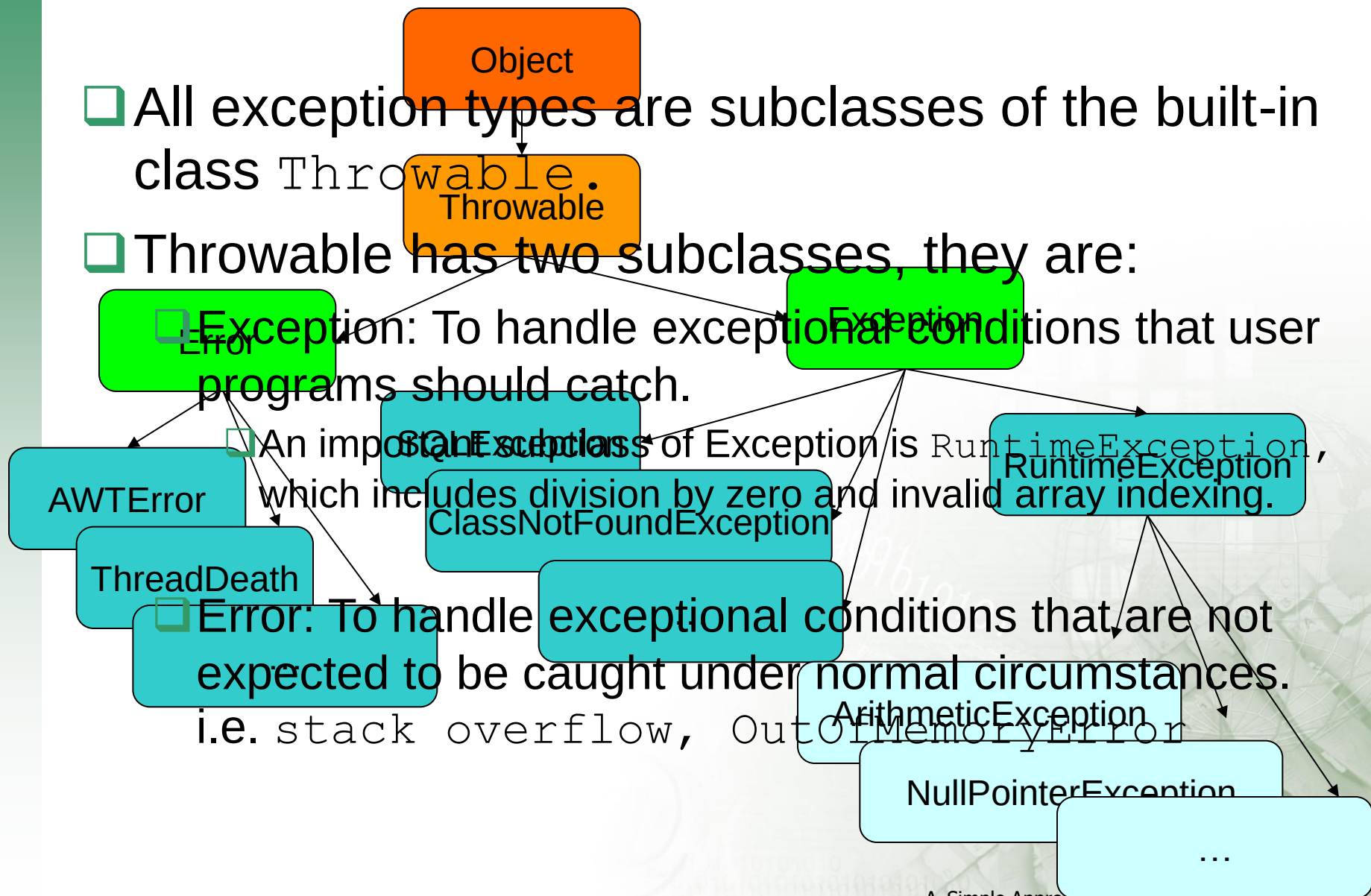
Handling Exceptions 2-2



Hierarchy of Exception classes 3-1



Hierarchy of Exception classes 3-1



Error vs Exceptions

- ❑ Error is used by the Java run-time system (JVM) to indicate errors that are associated with the run-time environment (JRE).
- ❑ StackOverflowError is an example of such an error.
- ❑ Whereas Exception is used for exceptional conditions that user programs should catch.
- ❑ NullPointerException is an example of such an exception.

Error vs Exceptions

Errors	Exceptions
Impossible to recover from an error	Possible to recover from exceptions
Errors are of type 'unchecked'	Exceptions can be either 'checked' or 'unchecked'
Occur at runtime	Can occur at compile time or run time
Caused by the application running environment	Caused by the application itself

Hierarchy of Exception classes 3-3

Exception	Description
Exception	Root class of exception hierarchy
RuntimeException	Base class for many java.lang exceptions
ArithmeticException	Arithmetic error condition, such as divide by zero
IllegalArgumentException	Method received illegal argument
ArrayIndexOutOfBoundsException	Array size is less or greater than actual array size
NullPointerException	Attempt to access <i>null</i> object member
SecurityException	Security settings do not allow operation
ClassNotFoundException	Unable to load requested class
NumberFormatException	Invalid conversion of a string to a numeric float
IOException	Root class for I/O exceptions
FileNotFoundException	Unable to locate a file
EOFException	End of file
IllegalAccessException	Access to a class denied
NoSuchMethodException	Requested method does not exist
InterruptedException	Thread interrupted

Types of Java Exceptions

- ❑ There are mainly two types of exceptions: checked and unchecked.
- ❑ Here, an error is considered as the unchecked exception.



Types of Exceptions

❑ Checked Exception

It is an exception that occurs at compile time, also called compile time exceptions.

❑ If some code within a method throws a checked exception, then the method must either handle the exception or it must specify the exception using *throws* keyword.

❑ The classes which directly inherit Throwable class except RuntimeException and Error are known as checked exceptions

❑ e.g. IOException, SQLException etc. Checked exceptions are checked at compile-time.

Types of Exceptions

❑ Unchecked Exception

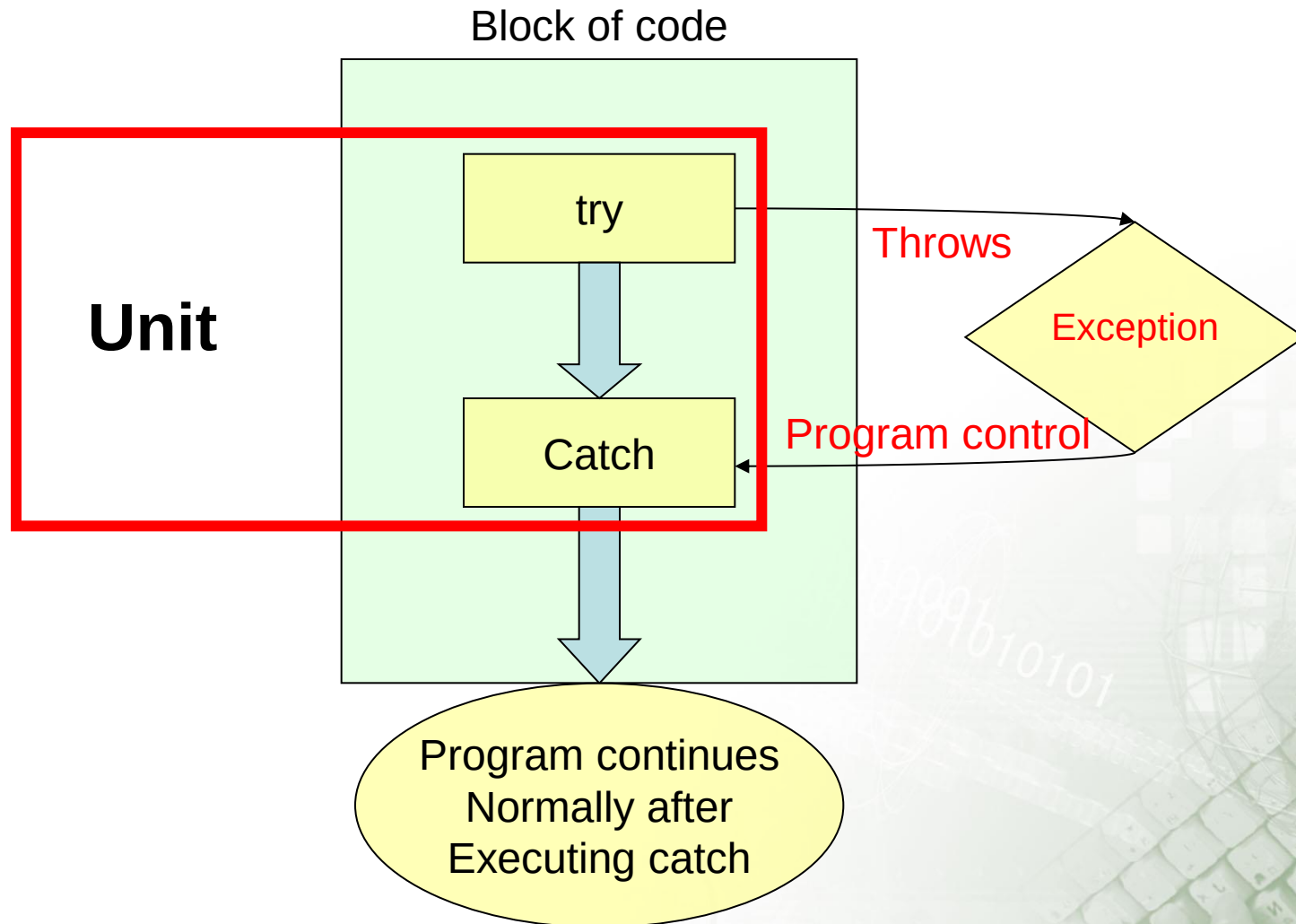
It is an exception that occurs at the time of execution.

- ❑ The classes which inherit RuntimeException are known as unchecked exceptions.
- ❑ e.g. ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException etc.
- ❑ Unchecked exceptions are not checked at compile-time, but they are checked at runtime.
- ❑ These are also called *Runtime Exceptions*.
- ❑ In C++, all exceptions are unchecked, so it is not forced by the compiler to either handle or specify the exception.
- ❑ It is up to the programmers to specify or catch the exceptions.

Error

- ❑ Error is irrecoverable
- ❑ e.g. `OutOfMemoryError`, `VirtualMachineError`, `AssertionError` etc.

try and catch blocks 2-1



```
import java.util.Scanner;
class Division {
public static void main(String[] args) {
    int a, b, result;
Scanner input = new Scanner(System.in);
    System.out.println("Input two integers");
    a = input.nextInt();
    b = input.nextInt();
result = a / b;
System.out.println("Result = " + result); } }
```

```
E:\Java>javac Division.java
```

```
E:\Java>java Division
```

```
Input two integers
```

```
4 2
```

```
Result = 2
```

```
E:\Java>java Division
```

```
Input two integers
```

```
7 0
```

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Division.main(Division.java:14)
```

```
E:\Java>_
```

Exception handling example

```
class Division {
public static void main(String[] args) {
    int a, b, result;
    Scanner input = new Scanner(System.in);
    System.out.println("Input two integers");
    a = input.nextInt();
    b = input.nextInt();
    try {
        result = a / b;
    } catch (ArithmeticException e) {
        System.out.println("Exception caught: Division by
zero.");
    } finally {input.close();
    System.out.println("In Finally Block");
    }
}
```

General Exception Handling

```
class ExceptionsDemo {
public static void main(String[] args) {
    String languages[] = { "C", "C++", "Java", "Perl",
"Python" };
    try {
for (int c = 1; c <= 5; c++) {
    System.out.println(languages[c]); } }
    catch (Exception e) {
System.out.println(e);
System.out.println(e.getMessage());
e.printStackTrace();
} } }
```

Different ways to print Exception messages in Java

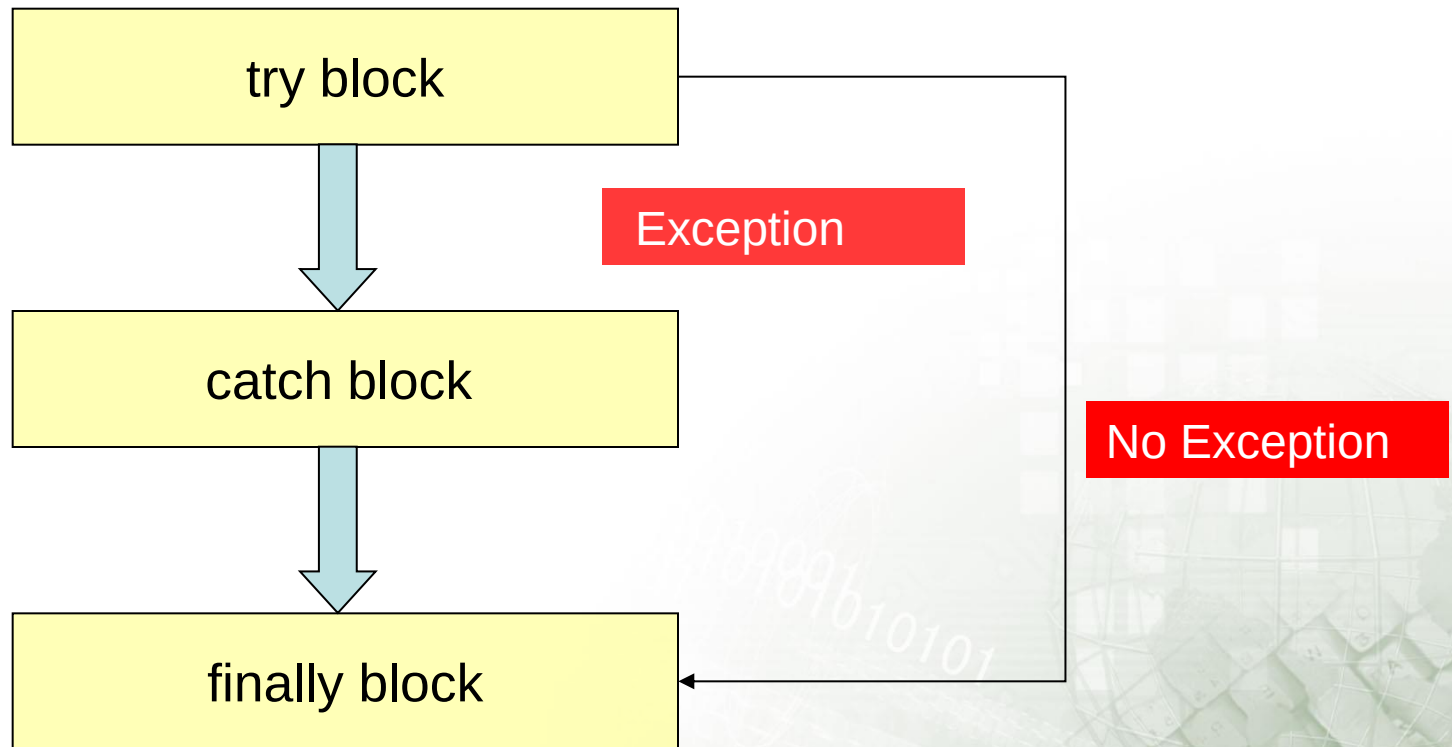
- ❑ **`java.lang.Throwable.printStackTrace()` method** : By using this method, we will get name(e.g. `java.lang.ArithmeticException`) and description(e.g. `/ by zero`) of an exception separated by colon, and stack trace (where in the code, that exception has occurred) in the next line.
- ❑ **`toString()` method** : By using this method, we will only get name and description of an exception. Note that this method is overridden in `Throwable` class.
- ❑ **`java.lang.Throwable.getMessage()` method** : By using this method, we will only get description of an exception.

try and catch blocks 2-2

```
class ArithmeticException {  
    /** Constructor. */  
    protected ArithmeticException() {  
    }  
    public static void main(final String[] args) {  
        ExceptionRaised obj = new ExceptionRaised();  
        try {  
            /* The variable result is define to  
            store the result. */  
            int result = obj.calculate(9, 0);  
            System.out.println(result);  
        } catch (Exception e) { // Exception  
            object  
            System.err.println("Exception occurred  :" +  
            e.toString());  
            e.printStackTrace();  
        }  
    }  
}
```

finally Block

Execution flow of try, catch and finally blocks



General form of exception-handling block

```
try{
    // block of code to monitor for errors
    methodGeneratingException();
}
catch (Exception e) {
    // exception handler for Exception e
}
finally{
    // block of code to be executed before
    try ends
    cleanup();
}
```

Multiple catch blocks

- ❑ Single piece of code can generate more than one error.
- ❑ When an exception is thrown, each `catch` statement is inspected in order, and the first one whose type matches that of the exception is executed.
- ❑ After one `catch` statement executes, the others are bypassed.

```
.....  
try{  
    }  
    catch (ArrayIndexOutOfBoundsException e) {  
    }  
    catch (Exception e) {  
    }  
..... •
```

Nested try - catch blocks

* All Rights Reserved This class demonstrate the nested try-catch statements.

```
* class NestedException {    /* Constructor. */
protected NestedException() {
} /** This method test the format of the number
 * @param argument is used to store the value of args.
 */
public test(String argumnet) {
    try {
        int num = args.length;
        /* Nested try block. */
        try {
            int numValue = Integer.parseInt(args[0]);
            System.out.println("The square of " + args[0] +
                + numValue * numValue);
        } catch (NumberFormatException nb) {
            /** Displaying the appropriate message, if exception
             * has occurred.
             */
            System.out.println("Not a number! ");
        }
        catch (ArrayIndexOutOfBoundsException ne) {
            System.out.println("Please enter the number!!!");
        }
    }
}

public static void main(final String[] args) {
    NestedException obj = new NestedException();
    obj.test(args[0]);
} }
```

Inner try
executed first



This is why
exception
handlers need
to be nested

Outer catch inspected in
case inner try does not
have a catch

Throw Keyword

- ❑ Throw keyword is used to throw an exception explicitly.
- ❑ Only object of Throwable class or its sub classes can be thrown.
- ❑ Program execution stops on encountering throw statement.
- ❑ The closest catch statement is checked for matching type of exception.

Syntax :

throw *ThrowableInstance*

Creating Instance of Throwable class

There are two possible ways to get an instance of class Throwable,

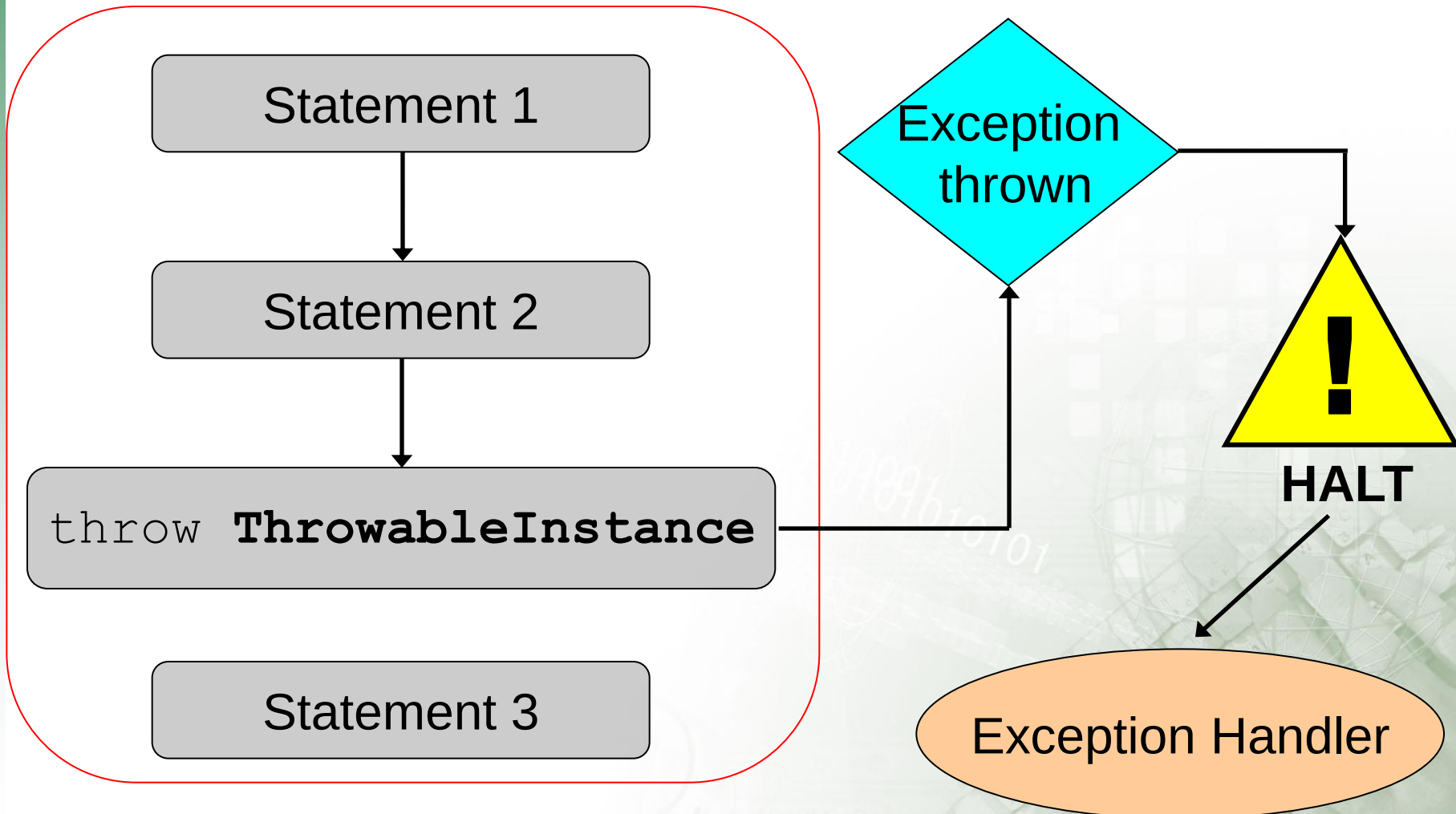
- ❑ Using a parameter in catch block.
- ❑ Creating instance with **new** operator.

```
new NullPointerException("test");
```

This constructs an instance of NullPointerException with name test.

Using throw & throws 2-1

Executable Program Statements



Demo throw Keyword

```
class Test {  
    static void avg() {  
        try {  
            throw new ArithmeticException("demo"); }  
        catch(ArithmeticException e)  
        {  
            System.out.println("Exception caught"); } }  
    public static void main(String args[]) {  
        avg();  
    } }  
}
```

avg() method throw an instance of ArithmeticException, which is successfully handled using the catch statement.

throws Keyword

- ❑ Any method capable of causing exceptions must list all the exceptions possible during its execution.
- ❑ So that anyone calling that method gets a prior knowledge about which exceptions to handle.
- ❑ A method can do so by using the **throws** keyword.
- ❑ **Throws keyword** is used for handling checked exceptions .
- ❑ By using throws we can declare multiple exceptions in one go.

Syntax :

```
type method_name(parameter_list) throws exception_list  
{  
    //definition of method  
}
```

Why Throws ?

- ❑ We already know we can handle exceptions using try-catch block.
- ❑ The throws does the same thing that try-catch does but there are some cases where you would prefer throws over try-catch.

For example:

- ❑ Lets say we have a method myMethod() that has statements that can throw either ArithmeticException or NullPointerException,
- ❑ In this case you can use try-catch block.

Multiple Exception Handling without Throws

```
public void myMethod()
{
    try {
        // Statements that might throw an exception
    }
    catch (ArithmeticException e) {
        // Exception handling statements
    }
    catch (NullPointerException e) {
        // Exception handling statements
    }
}
```

Why Throws?

- ❑ Suppose you have several such methods that can cause exceptions, in that case it would be tedious to write these try-catch for each method.
- ❑ The code will become unnecessary long and will be less-readable.
- ❑ One way to overcome this problem is by using throws like this: declare the exceptions in the method signature using throws and handle the exceptions where you are calling this method by using try-catch.
- ❑ Another advantage of using this approach is that you will be forced to handle the exception when you call this method
- ❑ All the exceptions that are declared using throws, must be handled where you are calling this method else you will get compilation error.

Exception Handling with Throws

```
public void myMethod() throws ArithmeticException, NullPointerException
```

```
{  
    // Statements that might throw an exception  
}
```

```
public static void main(String args[]) {  
    try {  
        myMethod();  
    }  
    catch (ArithmeticException e) {  
        // Exception handling statements  
    }  
    catch (NullPointerException e) {  
        // Exception handling statements  
    }  
}
```

Demo Throws

```
class Test {  
    static void check() throws ArithmeticException {  
        System.out.println("Inside check function");  
        throw new ArithmeticException("demo"); }  
    public static void main(String args[]) {  
        try {  
            check(); }  
            catch(ArithmeticException e) {  
                System.out.println("caught" + e);  
            } } }  
}
```

Throw vs Throws

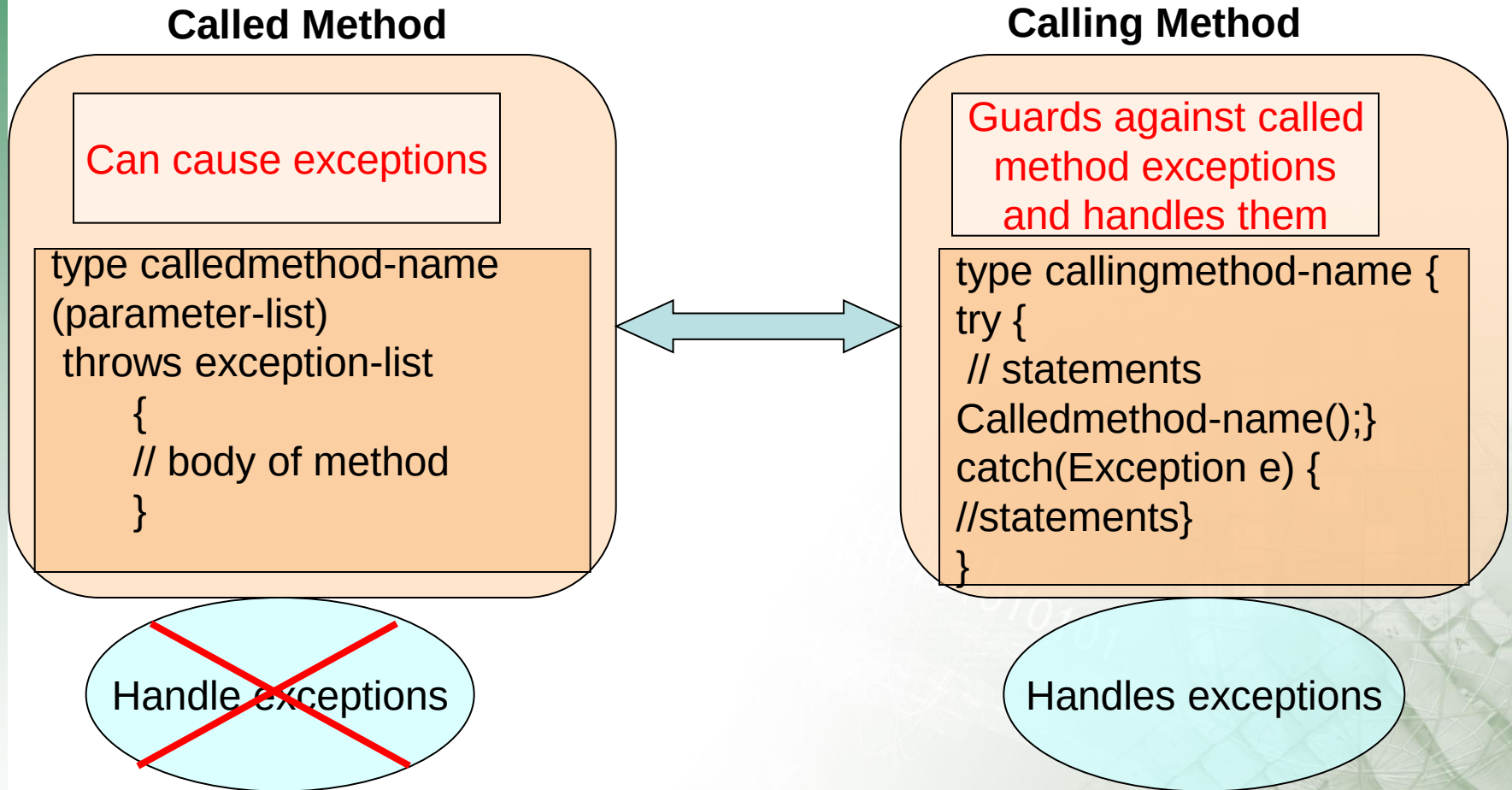
throw

1. Used to explicitly throw an exception
2. Checked exceptions cannot be propagated using throw only
3. Followed by an instance
4. Used within a method
5. Cannot throw multiple exceptions

throws

1. Used to declare an exception
2. Checked exceptions can be propagated
3. Followed by a class
4. Used with a method signature
5. Can declare multiple exceptions

Using throw & throws 2-2



User defined exceptions 2-1

Block of code

```
public class MyException extends Exception
{ public MyException() {}
  public MyException(String message) { super(message);
}
}
public static void main(String args[]) {
try {
throw new MyException
("Arg Length: " + args.length);
}
catch (MyException e) { e.printStackTrace(); } }
```

- Hence, User defined Exceptions Came into use.
- Subclass of exception class.
- Can use all methods of Throwable class.

Can generate exception which is not a part of built in exceptions.

User defined exceptions 2-2

```
class ExceptionClass {
ExceptionClass(final int val) {
    size = val;
    try {

class UserDefinedExceptions {
    /** Constructor. */
    protected UserDefinedExceptions() {
    }
    /**
     * Sole entry point to the class and application.
     * @param arg Array of String arguments.
     */
    public static void main(final String[] arg) {
        ExceptionClass obj = new
ExceptionClass(Integer.parseInt(arg[0]));
    }
}

    array = new int[3];
    for (int count = 0; count < 3; count++) {
        array[count] = count + 1;
    }
}
```

Summary

- ❑ Whenever an error is encountered during run time, an Exception occurs.
- ❑ A Java exception is an object that describes an exceptional condition that has occurred in a piece of code.
- ❑ When an exceptional condition arises, an object representing that exception is created and *thrown* in the method that caused the error.
- ❑ Java exception handling is managed using try, catch, throw, throws, and finally.
- ❑ Program statements to monitor are contained within a try block. Code within catch block catches the exception and handles it.
- ❑ Any code that absolutely must be executed before a method returns is put in a finally block.
- ❑ To manually throw an exception, use the keyword throw. Any exception that is thrown out of a method must be specified by a throws clause.